

使用 Material 3 建構動畫回應式應用程式的版面配置

來源：<https://codelabs.developers.google.com/codelabs/flutter-animated-responsive-layout?hl=zh-tw>

步驟數：9

在這個程式碼研究室中，您將瞭解如何建構採用自動調整式設計的應用程式，在 Flutter 支援的全部六個平台上，透過 Material 3 流暢地以動畫呈現效果。

目錄

1. [1. 簡介](#)
2. [2. 設定 Flutter 環境](#)
3. [3. 開始使用](#)
4. [4. 製作即時通訊應用程式](#)
5. [5. 新增 NavigationBar](#)
6. [6. 新增 NavigationRail](#)
7. [7. 為轉場效果加上動畫](#)
8. [8. 新增詳細資料檢視畫面清單](#)
9. [9. 恭喜](#)

步驟 1 · 約 0 分鐘

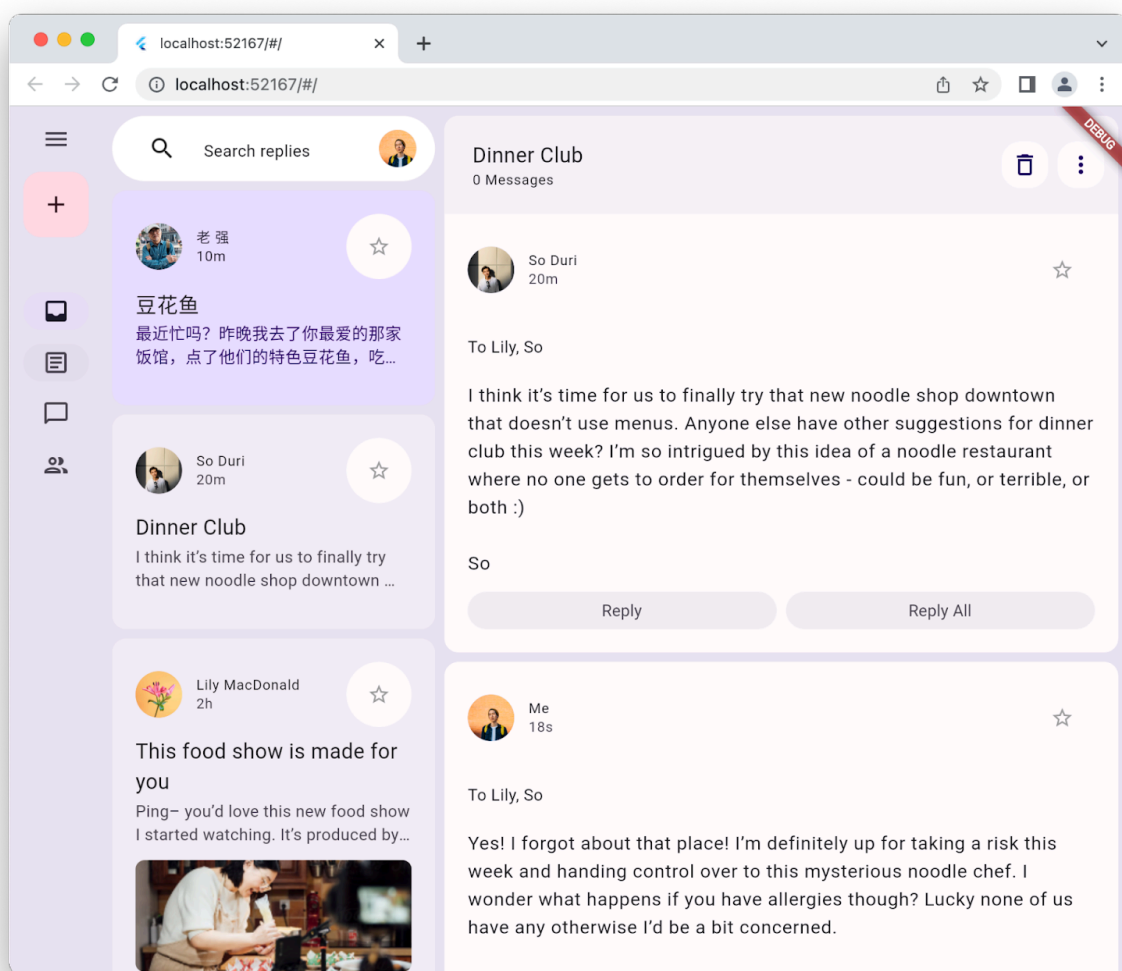
1. 簡介

Material 3 是最新版的 Google 開放原始碼設計系統。Flutter 持續擴大支援範圍，協助開發人員使用 Material 3 建構精美應用程式。在本程式碼研究室中，您將從空白的 Flutter 應用程式著手，使用 Flutter 的 Material 3 建構出具有完整樣式和動畫效果的應用程式。

建構項目

在本程式碼研究室中，您將建構模擬訊息應用程式。您的應用程式將會：

- 採用自適應設計，確保在電腦或行動裝置上都能正常運作。
- 使用動畫流暢地切換不同版面配置。
- 使用 Material 3 打造表現力十足的樣式。
- 在 Android、iOS、網頁、Windows、Linux 和 macOS 上執行。



本程式碼研究室著重於搭配 Flutter 使用 Material 3。我們不會對與主題無關的概念和程式碼多做介紹，但會事先準備好這些程式碼區塊，屆時您只要複製及貼上即可。

步驟 2 · 約 0 分鐘

2. 設定 Flutter 環境

軟硬體需求

- [Flutter SDK](#)
- 編輯器，例如 [VS Code](#) 或 [Android Studio](#)。

本程式碼研究室已測試過，可部署至 Android、iOS、網頁、Windows、Linux 和 macOS。部分部署目標需要安裝額外軟體，才能部署至該目標。如要瞭解平台設定是否正確，請執行 `flutter doctor`。

```
$ flutter doctor
Doctor summary (to see all details, run flutter doctor -v):
[✓] Flutter (Channel stable, 3.32.1, on macOS 15.5 24F74 darwin-arm64, locale en-AU)
[✓] Android toolchain - develop for Android devices (Android SDK version 36.0.0)
[✓] Xcode - develop for iOS and macOS (Xcode 16.3)
[✓] Chrome - develop for the web
[✓] Android Studio (version 2024.2)
[✓] IntelliJ IDEA Community Edition (version 2024.3.1.1)
[✓] VS Code (version 1.100.3)
[✓] Connected device (3 available)
[✓] Network resources

• No issues found!
```

如果輸出內容列出影響所選部署目標的問題，請執行 `flutter doctor -v`，取得更詳細的資訊。如果嘗試「`flutter doctor -v`」列出的步驟後仍無法解決問題，請考慮與 [Flutter 社群](#) 聯絡。

3. 開始使用

建立空白的 Flutter 應用程式

大多數 Flutter 開發人員都會使用 `flutter create` 建立「計算按鈕輕觸次數」的基本應用程式，然後花幾分鐘移除不需要的項目。您可以建立空白的 Flutter 專案 (使用 `--empty` 參數)，其中只包含讓應用程式運作的基本要素。

```
$ flutter create animated_responsive_layout --empty
Creating project animated_responsive_layout...
Resolving dependencies in `animated_responsive_layout`...
Downloading packages...
Got dependencies in `animated_responsive_layout`.
Wrote 129 files.

All done!
You can find general documentation for Flutter at: https://docs.flutter.dev/
Detailed API documentation is available at: https://api.flutter.dev/
If you prefer video documentation, consider: https://www.youtube.com/c/flutterdev

In order to run your empty application, type:

$ cd animated_responsive_layout
$ flutter run

Your empty application code is in animated_responsive_layout/lib/main.dart.
```

您可以透過程式碼編輯器或直接從指令列執行這段程式碼。系統可能會要求您決定要在哪個部署目標上執行應用程式，具體情況取決於您安裝的工具鍊，以及是否正在執行模擬器或模擬裝置。舉例來說，您可以選取「Chrome」選項，在網頁瀏覽器中執行空白應用程式，如下所示。

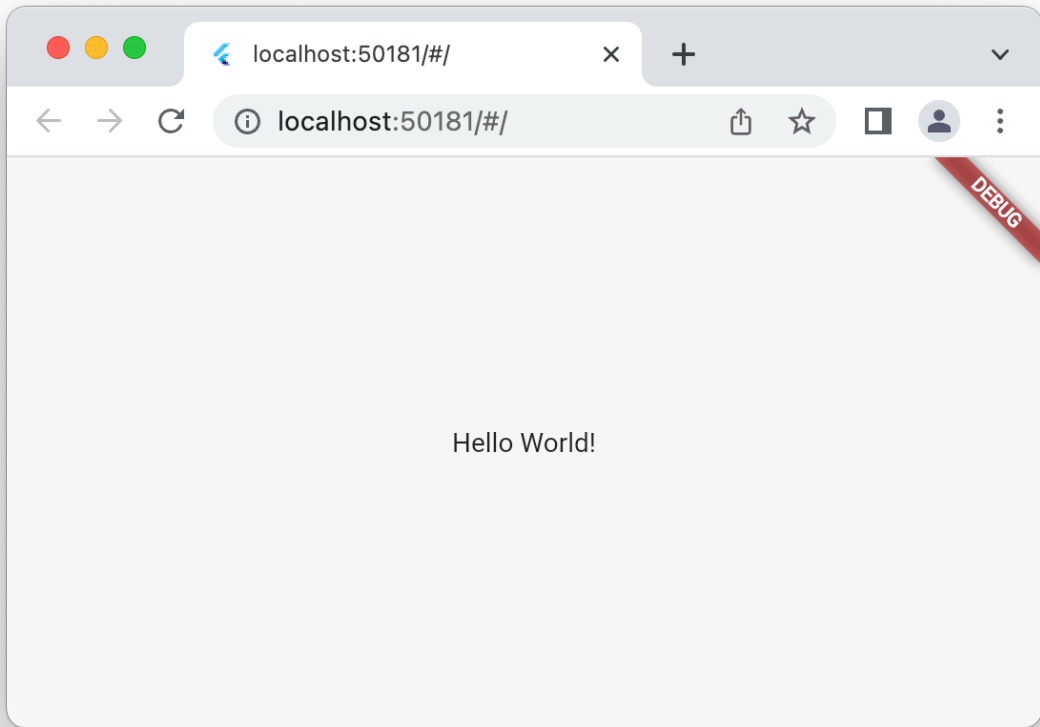
```
$ cd animated_responsive_layout
$ flutter run
Connected devices:
macOS (desktop) • macos • darwin-arm64 • macOS 15.5 24F74 darwin-arm64
Chrome (web) • chrome • web-javascript • Google Chrome 137.0.7151.56

[1]: macOS (macos)
[2]: Chrome (chrome)
Please choose one (or "q" to quit): 2
Launching lib/main.dart on Chrome in debug mode...
Waiting for connection from debug service on Chrome... 6.4s
This app is linked to the debug service: ws://127.0.0.1:60848/AM68Aq_ZiB8=/ws
Debug service listening on ws://127.0.0.1:60848/AM68Aq_ZiB8=/ws

Flutter run key commands.
R Hot restart.
h List all available interactive commands.
d Detach (terminate "flutter run" but leave application running).
c Clear the screen
q Quit (terminate the application on the device).

A Dart VM Service on Chrome is available at: http://127.0.0.1:60848/AM68Aq_ZiB8=
The Flutter DevTools debugger and profiler on Chrome is available at:
http://127.0.0.1:9100?uri=http://127.0.0.1:60848/AM68Aq_ZiB8=
Application finished.
```

在此情境中，您會在 Chrome 網路瀏覽器中看到空白應用程式正在執行。你也可以選擇在 Android、iOS 或電腦作業系統中執行。



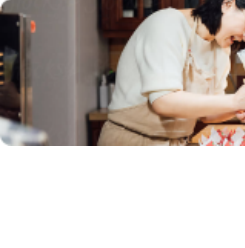
4. 製作即時通訊應用程式

建立虛擬化身

每個訊息應用程式都需要使用者圖片。這些圖片代表使用者，稱為「個人資料相片」。接著，在專案樹狀結構頂端建立 `assets` 目錄，並從[本程式碼研究室的 Git 存放區](#)填入一系列圖片。其中一種方式是使用 `wget` 指令列工具，如下所示。

```
$ mkdir assets
$ cd assets
$ for name in avatar_1 avatar_2 avatar_3 avatar_4 \
    avatar_5 avatar_6 avatar_7 thumbnail_1; \
do wget https://raw.githubusercontent.com/flutter/codelabs/main/animated-
responsive-layout/step_04/assets/$name.png ; \
done
```

這會將下列圖片下載到應用程式的 `assets` 目錄：

			
avatar_1.png	avatar_2.png	avatar_3.png	avatar_4.png
			
avatar_5.png	avatar_6.png	avatar_7.png	thumbnail_1.png

現在您已取得虛擬人偶圖片素材資源，請按照下列步驟將這些素材資源加入 `pubspec.yaml` 檔案：

pubspec.yaml


```
name: animated_responsive_layout
description: "A new Flutter project."
publish_to: 'none'
version: 0.1.0

environment:
  sdk: ^3.8.0

dependencies:
  flutter:
    sdk: flutter

dev_dependencies:
  flutter_test:
    sdk: flutter
  flutter_lints: ^5.0.0

flutter:
  uses-material-design: true

                                     # Add from here...

assets:
  - assets/avatar_1.png
  - assets/avatar_2.png
  - assets/avatar_3.png
  - assets/avatar_4.png
  - assets/avatar_5.png
  - assets/avatar_6.png
  - assets/avatar_7.png
  - assets/thumbnail_1.png

                                     # ... to here.
```

應用程式需要訊息的資料來源，在專案的 `lib` 目錄中，建立 `models` 子目錄。您可以在指令列中使用 `mkdir` 執行這項操作，也可以在選用的文字編輯器中執行。在 `lib/models` 目錄中建立 `models.dart` 檔案，並加入以下內容：

`lib/models/models.dart`

```

class Attachment {
  const Attachment({required this.url});

  final String url;
}

class Email {
  const Email({
    required this.sender,
    required this.recipients,
    required this.subject,
    required this.content,
    this.replies = 0,
    this.attachments = const [],
  });

  final User sender;
  final List<User> recipients;
  final String subject;
  final String content;
  final List<Attachment> attachments;
  final double replies;
}

class Name {
  const Name({required this.first, required this.last});

  final String first;
  final String last;
  String get fullName => '$first $last';
}

class User {
  const User({
    required this.name,
    required this.avatarUrl,
    required this.lastActive,
  });

  final Name name;
  final String avatarUrl;
  final DateTime lastActive;
}

```

現在您已定義資料的形狀，請在 `lib/models` 目錄中建立 `data.dart` 檔案，並加入下列內容：

[lib/models/data.dart](#)

```

import 'models.dart';

final User user_0 = User(
  name: const Name(first: 'Me', last: ''),
  avatarUrl: 'assets/avatar_1.png',
  lastActive: DateTime.now(),
);
final User user_1 = User(
  name: const Name(first: '老', last: '强'),
  avatarUrl: 'assets/avatar_2.png',
  lastActive: DateTime.now().subtract(const Duration(minutes: 10)),
);
final User user_2 = User(
  name: const Name(first: 'So', last: 'Duri'),
  avatarUrl: 'assets/avatar_3.png',
  lastActive: DateTime.now().subtract(const Duration(minutes: 20)),
);
final User user_3 = User(
  name: const Name(first: 'Lily', last: 'MacDonald'),
  avatarUrl: 'assets/avatar_4.png',
  lastActive: DateTime.now().subtract(const Duration(hours: 2)),
);
final User user_4 = User(
  name: const Name(first: 'Ziad', last: 'Aouad'),
  avatarUrl: 'assets/avatar_5.png',
  lastActive: DateTime.now().subtract(const Duration(hours: 6)),
);

final List<Email> emails = [
  Email(
    sender: user_1,
    recipients: [],
    subject: '豆花鱼',
    content: '最近忙吗？昨晚我去了你最爱的那家饭馆，点了他们的特色豆花鱼，吃着吃着就想你了。',
  ),
  Email(
    sender: user_2,
    recipients: [],
    subject: 'Dinner Club',
    content:
      'I think it's time for us to finally try that new noodle shop downtown that doesn't
      use menus. Anyone else have other suggestions for dinner club this week? I'm so intrigued by
      this idea of a noodle restaurant where no one gets to order for themselves - could be fun, or
      terrible, or both :)\n\nSo',
  ),
  Email(
    sender: user_3,
    recipients: [],
    subject: 'This food show is made for you',
    content:
      'Ping- you'd love this new food show I started watching. It's produced by a Thai
      drummer who started getting recognized for the amazing vegan food she always brought to

```

```

shows.',
  attachments: [const Attachment(url: 'assets/thumbnail_1.png')],
),
Email(
  sender: user_4,
  recipients: [],
  subject: 'Volunteer EMT with me?',
  content:
    'What do you think about training to be volunteer EMTs? We could do it together for
moral support. Think about it??',
),
];

final List<Email> replies = [
  Email(
    sender: user_2,
    recipients: [user_3, user_2],
    subject: 'Dinner Club',
    content:
      'I think it's time for us to finally try that new noodle shop downtown that doesn't
use menus. Anyone else have other suggestions for dinner club this week? I'm so intrigued by
this idea of a noodle restaurant where no one gets to order for themselves - could be fun, or
terrible, or both :)\\n\\nSo',
  ),
  Email(
    sender: user_0,
    recipients: [user_3, user_2],
    subject: 'Dinner Club',
    content:
      'Yes! I forgot about that place! I'm definitely up for taking a risk this week and
handing control over to this mysterious noodle chef. I wonder what happens if you have
allergies though? Lucky none of us have any otherwise I'd be a bit concerned.\\n\\nThis is
going to be great. See you all at the usual time?',
  ),
];

```

有了這些資料後，就可以定義幾個小工具來顯示資料。在 `lib` 下方建立名為 `widgets` 的子目錄。您會在 `widgets` 中建立四個檔案，在建立所有檔案前，編輯器可能會顯示一些警告。請注意，本程式碼研究室的重點是使用 Material 3 設定應用程式樣式。因此，請新增下列四個檔案，並加入所列內容：

[lib/widgets/email_list_view.dart](#)

```

import 'package:flutter/material.dart';

import '../models/data.dart' as data;
import '../models/models.dart';
import 'email_widget.dart';
import 'search_bar.dart' as search_bar;

class EmailListView extends StatelessWidget {
  const EmailListView({
    super.key,
    this.selectedIndex,
    this.onSelected,
    required this.currentUser,
  });

  final int? selectedIndex;
  final ValueChanged<int>? onSelected;
  final User currentUser;

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.symmetric(horizontal: 8.0),
      child: ListView(
        children: [
          const SizedBox(height: 8),
          search_bar.SearchBar(currentUser: currentUser),
          const SizedBox(height: 8),
          ...List.generate(data.emails.length, (index) {
            return Padding(
              padding: const EdgeInsets.only(bottom: 8.0),
              child: EmailWidget(
                email: data.emails[index],
                onSelected: onSelected != null
                  ? () {
                      onSelected!(index);
                    }
                  : null,
                isSelected: selectedIndex == index,
              ),
            );
          }),
        ],
      ),
    );
  }
}

```

顯示電子郵件清單似乎是訊息應用程式應具備的功能。編輯器會提出幾項抱怨，但您可以新增下一個檔案 `email_widget.dart`，修正其中一些問題。

[lib/widgets/email_widget.dart](#)

```

import 'package:flutter/material.dart';
import '../models/models.dart';
import 'star_button.dart';

enum EmailType { preview, threaded, primaryThreaded }

class EmailWidget extends StatefulWidget {
  const EmailWidget({
    super.key,
    required this.email,
    this.isSelected = false,
    this.isPreview = true,
    this.isThreaded = false,
    this.showHeadline = false,
    this.onSelected,
  });

  final bool isSelected;
  final bool isPreview;
  final bool showHeadline;
  final bool isThreaded;
  final void Function()? onSelected;
  final Email email;

  @override
  State<EmailWidget> createState() => _EmailWidgetState();
}

class _EmailWidgetState extends State<EmailWidget> {
  late final ColorScheme _colorScheme = Theme.of(context).colorScheme;
  late Color unselectedColor = Color.alphaBlend(
    _colorScheme.primary.withAlpha(20),
    _colorScheme.surface,
  );

  Color get _surfaceColor => switch (widget) {
    EmailWidget(isPreview: false) => _colorScheme.surface,
    EmailWidget(isSelected: true) => _colorScheme.primaryContainer,
    _ => unselectedColor,
  };

  @override
  Widget build(BuildContext context) {
    return GestureDetector(
      onTap: widget.onSelected,
      child: Card(
        elevation: 0,
        color: _surfaceColor,
        clipBehavior: Clip.hardEdge,
        child: Column(
          crossAxisAlignment: CrossAxisAlignment.start,
          mainAxisAlignment: MainAxisAlignment.min,

```

```

        children: [
          if (widget.showHeadline) ...[
            EmailHeadline(email: widget.email, isSelected: widget.isSelected),
          ],
          EmailContent(
            email: widget.email,
            isPreview: widget.isPreview,
            isThreaded: widget.isThreaded,
            isSelected: widget.isSelected,
          ),
        ],
      ),
    ),
  );
}
}

```

```

class EmailContent extends StatefulWidget {
  const EmailContent({
    super.key,
    required this.email,
    required this.isPreview,
    required this.isThreaded,
    required this.isSelected,
  });

  final Email email;
  final bool isPreview;
  final bool isThreaded;
  final bool isSelected;

  @override
  State<EmailContent> createState() => _EmailContentState();
}

```

```

class _EmailContentState extends State<EmailContent> {
  late final ColorScheme _colorScheme = Theme.of(context).colorScheme;
  late final TextTheme _textTheme = Theme.of(context).textTheme;

  Widget get contentSpacer => SizedBox(height: widget.isThreaded ? 20 : 2);

  String get lastActiveLabel {
    final DateTime now = DateTime.now();
    if (widget.email.sender.lastActive.isAfter(now)) throw ArgumentError();
    final Duration elapsedTime = widget.email.sender.lastActive
      .difference(now)
      .abs();
    return switch (elapsedTime) {
      Duration(inSeconds: < 60) => '${elapsedTime.inSeconds}s',
      Duration(inMinutes: < 60) => '${elapsedTime.inMinutes}m',
      Duration(inHours: < 24) => '${elapsedTime.inHours}h',
      Duration(inDays: < 365) => '${elapsedTime.inDays}d',
    };
  }
}

```



```

    _ => throw UnimplementedError(),
  };
}

TextStyle? get contentTextStyle => switch (widget) {
  EmailContent(isThreaded: true) => _textTheme.bodyLarge,
  EmailContent(isSelected: true) => _textTheme.bodyMedium?.copyWith(
    color: _colorScheme.onPrimaryContainer,
  ),
  _ => _textTheme.bodyMedium?.copyWith(color: _colorScheme.onSurfaceVariant),
};

@override
Widget build(BuildContext context) {
  return Padding(
    padding: const EdgeInsets.all(20),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        LayoutBuilder(
          builder: (context, constraints) {
            return Row(
              crossAxisAlignment: CrossAxisAlignment.center,
              children: [
                if (constraints.maxWidth - 200 > 0) ...[
                  CircleAvatar(
                    backgroundImage: AssetImage(
                      widget.email.sender.avatarUrl,
                    ),
                  ),
                ],
                const Padding(
                  padding: EdgeInsets.symmetric(horizontal: 6.0),
                ),
              ],
            ),
            Expanded(
              child: Column(
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text(
                    widget.email.sender.name.fullName,
                    overflow: TextOverflow.fade,
                    maxLines: 1,
                    style: widget.isSelected
                      ? _textTheme.labelMedium?.copyWith(
                          color: _colorScheme.onSecondaryContainer,
                        )
                      : _textTheme.labelMedium?.copyWith(
                          color: _colorScheme.onSurface,
                        ),
                  ),
                  Text(
                    lastActiveLabel,

```

```

        overflow: TextOverflow.fade,
        maxLines: 1,
        style: widget.isSelected
          ? _textTheme.labelMedium?.copyWith(
              color: _colorScheme.onSecondaryContainer,
            )
          : _textTheme.labelMedium?.copyWith(
              color: _colorScheme.onSurfaceVariant,
            ),
      ),
    ],
  ),
  if (constraints.maxWidth - 200 > 0) ...[const StarButton()],
],
);
},
),
const SizedBox(width: 8),
Column(
  crossAxisAlignment: CrossAxisAlignment.start,
  children: [
    if (widget.isPreview) ...[
      Text(
        widget.email.subject,
        style: const TextStyle(
          fontSize: 18,
        ).copyWith(color: _colorScheme.onSurface),
      ),
    ],
    if (widget.isThreaded) ...[
      contentSpacer,
      Text(
        "To ${widget.email.recipients.map((recipient) =>
recipient.name.first).join(", ")}",
        style: _textTheme.bodyMedium,
      ),
    ],
    contentSpacer,
    Text(
      widget.email.content,
      maxLines: widget.isPreview ? 2 : 100,
      overflow: TextOverflow.ellipsis,
      style: contentTextStyle,
    ),
  ],
),
const SizedBox(width: 12),
widget.email.attachments.isNotEmpty
  ? Container(
      height: 96,
      decoration: BoxDecoration(

```

```

        borderRadius: BorderRadius.circular(8.0),
        image: DecorationImage(
          fit: BoxFit.cover,
          image: AssetImage(widget.email.attachments.first.url),
        ),
      ),
    ),
    : const SizedBox.shrink(),
    if (!widget.isPreview) ...[const EmailReplyOptions()],
  ],
),
);
}
}

```

```

class EmailHeadline extends StatefulWidget {
  const EmailHeadline({
    super.key,
    required this.email,
    required this.isSelected,
  });

  final Email email;
  final bool isSelected;

  @override
  State<EmailHeadline> createState() => _EmailHeadlineState();
}

```

```

class _EmailHeadlineState extends State<EmailHeadline> {
  late final TextTheme _textTheme = Theme.of(context).textTheme;
  late final ColorScheme _colorScheme = Theme.of(context).colorScheme;

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        return Container(
          height: 84,
          color: Color.alphaBlend(
            _colorScheme.primary.withAlpha(12),
            _colorScheme.surface,
          ),
          child: Padding(
            padding: const EdgeInsets.fromLTRB(24, 12, 12, 12),
            child: Row(
              mainAxisAlignment: MainAxisAlignment.max,
              crossAxisAlignment: CrossAxisAlignment.center,
              children: [
                Expanded(
                  child: Column(
                    mainAxisAlignment: MainAxisAlignment.min,

```

```

crossAxisAlignment: CrossAxisAlignment.start,
children: [
  Text(
    widget.email.subject,
    maxLines: 1,
    overflow: TextOverflow.fade,
    style: const TextStyle(
      fontSize: 18,
      fontWeight: FontWeight.w400,
    ),
  ),
  Text(
    '${widget.email.replies.toString()} Messages',
    maxLines: 1,
    overflow: TextOverflow.fade,
    style: _textTheme.labelMedium?.copyWith(
      fontWeight: FontWeight.w500,
    ),
  ),
],
),
),
// Display a "condensed" version if the widget in the row are
// expected to overflow.
if (constraints.maxWidth - 200 > 0) ...[
  SizedBox(
    height: 40,
    width: 40,
    child: FloatingActionButton(
      onPressed: () {},
      elevation: 0,
      backgroundColor: _colorScheme.surface,
      child: const Icon(Icons.delete_outline),
    ),
  ),
  const Padding(padding: EdgeInsets.only(right: 8.0)),
  SizedBox(
    height: 40,
    width: 40,
    child: FloatingActionButton(
      onPressed: () {},
      elevation: 0,
      backgroundColor: _colorScheme.surface,
      child: const Icon(Icons.more_vert),
    ),
  ),
],
),
),
);
},

```

```

    );
  }
}

class EmailReplyOptions extends StatefulWidget {
  const EmailReplyOptions({super.key});

  @override
  State<EmailReplyOptions> createState() => _EmailReplyOptionsState();
}

class _EmailReplyOptionsState extends State<EmailReplyOptions> {
  late final ColorScheme _colorScheme = Theme.of(context).colorScheme;

  @override
  Widget build(BuildContext context) {
    return LayoutBuilder(
      builder: (context, constraints) {
        if (constraints.maxWidth < 100) {
          return const SizedBox.shrink();
        }
        return Row(
          children: [
            Expanded(
              child: TextButton(
                style: ButtonStyle(
                  backgroundColor: WidgetStateProperty.all(
                    _colorScheme.onInverseSurface,
                  ),
                ),
                onPressed: () {},
                child: Text(
                  'Reply',
                  style: TextStyle(color: _colorScheme.onSurfaceVariant),
                ),
              ),
            const SizedBox(width: 8),
            Expanded(
              child: TextButton(
                style: ButtonStyle(
                  backgroundColor: WidgetStateProperty.all(
                    _colorScheme.onInverseSurface,
                  ),
                ),
                onPressed: () {},
                child: Text(
                  'Reply All',
                  style: TextStyle(color: _colorScheme.onSurfaceVariant),
                ),
              ),
            ),
          ],
        );
      },
    );
  }
}

```

```
        ],  
      );  
    },  
  );  
}  
}
```

是的，該小工具會顯示許多資訊。建議您詳細研究，特別是瞭解如何在整個小工具中套用顏色。這會成為重複出現的主題。下一則是「`search_bar.dart`」。

[lib/widgets/search_bar.dart](#)

```

import 'package:flutter/material.dart';

import '../models/models.dart';

class SearchBar extends StatelessWidget {
  const SearchBar({super.key, required this.currentUser});

  final User currentUser;

  @override
  Widget build(BuildContext context) {
    return SizedBox(
      height: 56,
      child: Container(
        decoration: BoxDecoration(
          borderRadius: BorderRadius.circular(100),
          color: Colors.white,
        ),
        padding: const EdgeInsets.fromLTRB(31, 12, 12, 12),
        child: Row(
          crossAxisAlignment: CrossAxisAlignment.center,
          children: [
            const Icon(Icons.search),
            const SizedBox(width: 23.5),
            Expanded(
              child: TextField(
                maxLines: 1,
                decoration: InputDecoration(
                  isDense: true,
                  border: InputBorder.none,
                  hintText: 'Search replies',
                  hintStyle: Theme.of(context).textTheme.bodyMedium,
                ),
              ),
            ),
            CircleAvatar(backgroundImage: AssetImage(currentUser.avatarUrl)),
          ],
        ),
      ),
    );
  }
}

```

這個小工具簡單許多，而且是無狀態。接著新增另一個小工具「`star_button.dart`」：

[lib/widgets/star_button.dart](#)

```

import 'package:flutter/material.dart';

class StarButton extends StatefulWidget {
  const StarButton({super.key});

  @override
  State<StarButton> createState() => _StarButtonState();
}

class _StarButtonState extends State<StarButton> {
  bool state = false;
  late final ColorScheme _colorScheme = Theme.of(context).colorScheme;

  Icon get icon {
    final IconData iconData = state ? Icons.star : Icons.star_outline;

    return Icon(iconData, color: Colors.grey, size: 20);
  }

  void _toggle() {
    setState(() {
      state = !state;
    });
  }

  double get turns => state ? 1 : 0;

  @override
  Widget build(BuildContext context) {
    return AnimatedRotation(
      turns: turns,
      curve: Curves.decelerate,
      duration: const Duration(milliseconds: 300),
      child: FloatingActionButton(
        elevation: 0,
        shape: const CircleBorder(),
        backgroundColor: _colorScheme.surface,
        onPressed: () => _toggle(),
        child: Padding(padding: const EdgeInsets.all(10.0), child: icon),
      ),
    );
  }
}

```

接著，更新節目主角 `lib/main.dart`。將該檔案的現有內容替換成下列內容。

[lib/main.dart](#)


```

import 'package:flutter/material.dart';

import 'models/data.dart' as data;
import 'models/models.dart';
import 'widgets/email_list_view.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData.light(),
      home: Feed(currentUser: data.user_0),
    );
  }
}

class Feed extends StatefulWidget {
  const Feed({super.key, required this.currentUser});

  final User currentUser;

  @override
  State<Feed> createState() => _FeedState();
}

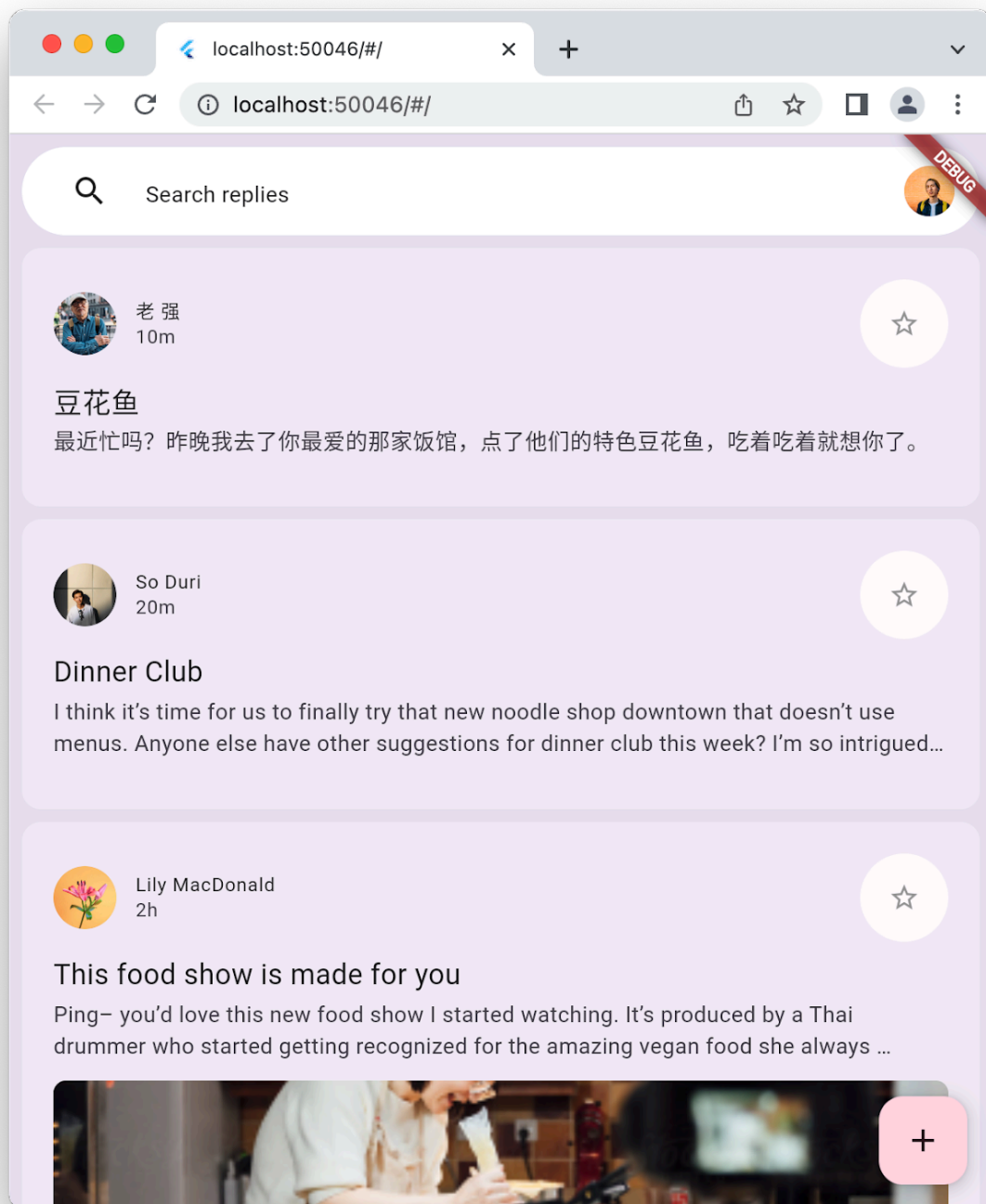
class _FeedState extends State<Feed> {
  late final _colorScheme = Theme.of(context).colorScheme;
  late final _backgroundColor = Color.alphaBlend(
    _colorScheme.primary.withAlpha(36),
    _colorScheme.surface,
  );

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Container(
        color: _backgroundColor,
        child: EmailListView(currentUser: widget.currentUser),
      ),
      floatingActionButton: FloatingActionButton(
        backgroundColor: _colorScheme.tertiaryContainer,
        foregroundColor: _colorScheme.onTertiaryContainer,
        onPressed: () {},
        child: const Icon(Icons.add),
      ),
    );
  }
}

```

```
}  
}
```

執行應用程式，看看要從什麼地方開始。



5. 新增 `NavigationBar`

在上一個步驟的結尾，啟動器應用程式會顯示訊息清單，但沒有其他活動。在這個步驟中，您會新增 `NavigationBar`，增添視覺趣味。隨著應用程式從 UI 草圖演變成實際應用程式，導覽列會提供應用程式的不同區域供使用者使用。

有 `NavigationBar` 表示有可導覽到的目的地。在 `lib` 目錄中新建名為 `destinations.dart` 的檔案，並填入下列程式碼。

`lib/destinations.dart`

```
import 'package:flutter/material.dart';

class Destination {
  const Destination(this.icon, this.label);
  final IconData icon;
  final String label;
}

const List<Destination> destinations = <Destination>[
  Destination(Icons.inbox_rounded, 'Inbox'),
  Destination(Icons.article_outlined, 'Articles'),
  Destination(Icons.messenger_outline_rounded, 'Messages'),
  Destination(Icons.group_outlined, 'Groups'),
];
```

這樣一來，應用程式就會有四個目的地可供 `NavigationBar` 顯示。接著，請按照下列方式將目的地清單連線至

`lib/main.dart` 檔案：

`lib/main.dart`

```

import 'package:flutter/material.dart';

import 'destinations.dart';           // Add this import
import 'models/data.dart' as data;
import 'models/models.dart';
import 'widgets/email_list_view.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData.light(),
      home: Feed(currentUser: data.user_0),
    );
  }
}

class Feed extends StatefulWidget {
  const Feed({super.key, required this.currentUser});

  final User currentUser;

  @override
  State<Feed> createState() => _FeedState();
}

class _FeedState extends State<Feed> {
  late final _colorScheme = Theme.of(context).colorScheme;
  late final _backgroundColor = Color.alphaBlend(
    _colorScheme.primary.withAlpha(36),
    _colorScheme.surface,
  );

  int selectedIndex = 0;           // Add this variable

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Container(
        color: _backgroundColor,
        child: EmailListView(
          // Add from here...
          selectedIndex: selectedIndex,
          onSelect: (index) {
            setState(() {
              selectedIndex = index;
            });
          },
        ),
      ),
    );
  }
}

```

```

        });
    },
    // ... to here.

    currentUser: widget.currentUser,
  ),
),
floatingActionButton: FloatingActionButton(
  backgroundColor: _colorScheme.tertiaryContainer,
  foregroundColor: _colorScheme.onTertiaryContainer,
  onPressed: () {},
  child: const Icon(Icons.add),
),
// Add from here...

bottomNavigationBar: NavigationBar(
  elevation: 0,
  backgroundColor: Colors.white,
  destinations: destinations.map<NavigationDestination>((d) {
    return NavigationDestination(icon: Icon(d.icon), label: d.label);
  }).toList(),
  selectedIndex: selectedIndex,
  onDestinationSelected: (index) {
    setState(() {
      selectedIndex = index;
    });
  },
),
// ...to here.

);
}
}

```

您不必為每個目的地定義不同內容，只要變更個別訊息的狀態，即可在 `NavigationBar` 中反映所選目的地。為確保一致性，反向操作也適用：選取訊息時，`NavigationBar` 會顯示對應目的地。執行應用程式，確認這些變更：



在窄版設定中，這看起來很合理，但如果將視窗變寬，或將手機模擬器旋轉為橫向，就會有點奇怪。如要修正這個問題，請在應用程式寬度足夠時，在畫面左側導入 `NavigationRail`。這會在下一個步驟中處理。

6. 新增 NavigationRail

這個步驟會在應用程式中新增 `NavigationRail`。概念是根據螢幕大小只顯示兩個導覽小工具中的其中一個，也就是說，您需要視需要隱藏或顯示 `NavigationBar`。在 `lib/widgets` 目錄中，建立 `disappearing_bottom_navigation_bar.dart` 檔案並加入下列程式碼：

`lib/widgets/disappearing_bottom_navigation_bar.dart`

```
import 'package:flutter/material.dart';

import '../destinations.dart';

class DisappearingBottomNavigationBar extends StatelessWidget {
  const DisappearingBottomNavigationBar({
    super.key,
    required this.selectedIndex,
    this.onDestinationSelected,
  });

  final int selectedIndex;
  final ValueChanged<int>? onDestinationSelected;

  @override
  Widget build(BuildContext context) {
    return NavigationBar(
      elevation: 0,
      backgroundColor: Colors.white,
      destinations: destinations.map<NavigationDestination>((d) {
        return NavigationDestination(icon: Icon(d.icon), label: d.label);
      }).toList(),
      selectedIndex: selectedIndex,
      onDestinationSelected: onDestinationSelected,
    );
  }
}
```

在同一個目錄中，新增另一個名為 `disappearing_navigation_rail.dart` 的檔案，並加入下列程式碼：

`lib/widgets/disappearing_navigation_rail.dart`

```

import 'package:flutter/material.dart';

import '../destinations.dart';

class DisappearingNavigationRail extends StatelessWidget {
  const DisappearingNavigationRail({
    super.key,
    required this.backgroundColor,
    required this.selectedIndex,
    this.onDestinationSelected,
  });

  final Color backgroundColor;
  final int selectedIndex;
  final ValueChanged<int>? onDestinationSelected;

  @override
  Widget build(BuildContext context) {
    final colorScheme = Theme.of(context).colorScheme;
    return NavigationRail(
      selectedIndex: selectedIndex,
      backgroundColor: backgroundColor,
      onDestinationSelected: onDestinationSelected,
      leading: Column(
        children: [
          IconButton(onPressed: () {}, icon: const Icon(Icons.menu)),
          const SizedBox(height: 8),
          FloatingActionButton(
            shape: const RoundedRectangleBorder(
              borderRadius: BorderRadius.all(Radius.circular(15)),
            ),
            backgroundColor: colorScheme.tertiaryContainer,
            foregroundColor: colorScheme.onTertiaryContainer,
            onPressed: () {},
            child: const Icon(Icons.add),
          ),
        ],
      ),
      groupAlignment: -0.85,
      destinations: destinations.map((d) {
        return NavigationRailDestination(
          icon: Icon(d.icon),
          label: Text(d.label),
        );
      }).toList(),
    );
  }
}

```

導覽慣用語重構為專屬小工具後，`lib/main.dart` 檔案需要進行一些修改：


```

import 'package:flutter/material.dart';

// Remove the destination.dart import, it's not required
import 'models/data.dart' as data;
import 'models/models.dart';
import 'widgets/disappearing_bottom_navigation_bar.dart'; // Add import
import 'widgets/disappearing_navigation_rail.dart';        // Add import
import 'widgets/email_list_view.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData.light(),
      home: Feed(currentUser: data.user_0),
    );
  }
}

class Feed extends StatefulWidget {
  const Feed({super.key, required this.currentUser});

  final User currentUser;

  @override
  State<Feed> createState() => _FeedState();
}

class _FeedState extends State<Feed> {
  late final _colorScheme = Theme.of(context).colorScheme;
  late final _backgroundColor = Color.alphaBlend(
    _colorScheme.primary.withAlpha(36),
    _colorScheme.surface,
  );

  int selectedIndex = 0;

  bool wideScreen = false;

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();

    final double width = MediaQuery.of(context).size.width;
    wideScreen = width > 600;
  }
}

```

// ... to here.

```
@override
Widget build(BuildContext context) {

    // Modify from here...

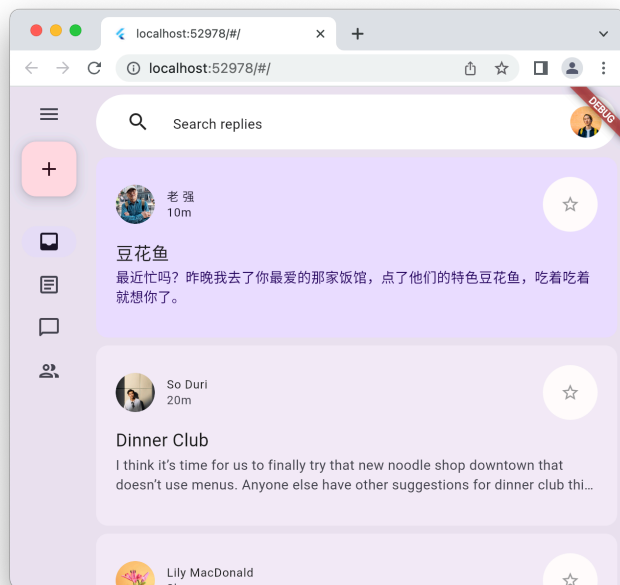
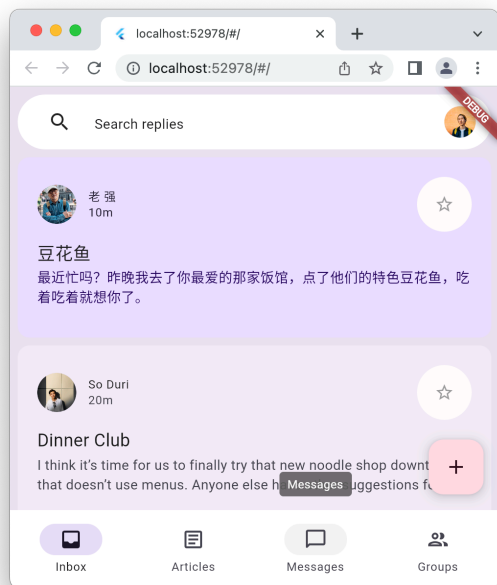
    return Scaffold(
      body: Row(
        children: [
          if (wideScreen)
            DisappearingNavigationRail(
              selectedIndex: selectedIndex,
              backgroundColor: _backgroundColor,
              onDestinationSelected: (index) {
                setState(() {
                  selectedIndex = index;
                });
              },
            ),
          Expanded(
            child: Container(
              color: _backgroundColor,
              child: EmailListView(
                selectedIndex: selectedIndex,
                onSelected: (index) {
                  setState(() {
                    selectedIndex = index;
                  });
                },
                currentUser: widget.currentUser,
              ),
            ),
          ),
        ],
      ),
      floatingActionButton: wideScreen
        ? null
        : FloatingActionButton(
            backgroundColor: _colorScheme.tertiaryContainer,
            foregroundColor: _colorScheme.onTertiaryContainer,
            onPressed: () {},
            child: const Icon(Icons.add),
          ),
      bottomNavigationBar: wideScreen
        ? null
        : DisappearingBottomNavigationBar(
            selectedIndex: selectedIndex,
            onDestinationSelected: (index) {
              setState(() {
                selectedIndex = index;
              });
            },
          ),
    );
}
```

```
);

// ... to here.

}
}
```

`main.dart` 檔案的第一個重要變更，是新增 `wideScreen` 狀態，每當使用者調整螢幕大小 (無論是調整瀏覽器視窗大小或旋轉手機)，這個狀態就會更新。下一個變更會修改 `NavigationBar` 和 `FloatingActionButton`，視應用程式是否處於 `wideScreen` 模式而定。最後，如果螢幕夠寬，系統會在左側有條件地顯示 `NavigationRail`。在網頁或桌面上執行應用程式，然後調整畫面大小，顯示兩種不同的版面配置。



有兩種不同的版面配置很好，但兩者之間的轉場效果不佳。以更動態的方式將導覽列換成導覽邊欄 (反之亦然)，可大幅改善這個應用程式。您將在下一個步驟中新增這項動畫。

步驟 7 · 約 15 分鐘

7. 為轉場效果加上動畫

如要建立動畫體驗，需要製作一系列動畫，並適當編排每個元件的動畫。如要製作這個動畫，請先在 `lib` 目錄中建立名為 `animations.dart` 的新檔案，並加入所需的動畫曲線。

[lib/animations.dart](#)

```

import 'package:flutter/animation.dart';

class BarAnimation extends ReverseAnimation {
  BarAnimation({required AnimationController parent})
    : super(
        CurvedAnimation(
          parent: parent,
          curve: const Interval(0, 1 / 5),
          reverseCurve: const Interval(1 / 5, 4 / 5),
        ),
      );
}

class OffsetAnimation extends CurvedAnimation {
  OffsetAnimation({required super.parent})
    : super(
        curve: const Interval(
          2 / 5,
          3 / 5,
          curve: Curves.easeInOutCubicEmphasized,
        ),
        reverseCurve: Interval(
          4 / 5,
          1,
          curve: Curves.easeInOutCubicEmphasized.flipped,
        ),
      );
}

class RailAnimation extends CurvedAnimation {
  RailAnimation({required super.parent})
    : super(
        curve: const Interval(0 / 5, 4 / 5),
        reverseCurve: const Interval(3 / 5, 1),
      );
}

class RailFabAnimation extends CurvedAnimation {
  RailFabAnimation({required super.parent})
    : super(curve: const Interval(3 / 5, 1));
}

class ScaleAnimation extends CurvedAnimation {
  ScaleAnimation({required super.parent})
    : super(
        curve: const Interval(
          3 / 5,
          4 / 5,
          curve: Curves.easeInOutCubicEmphasized,
        ),
        reverseCurve: Interval(
          3 / 5,

```

```

        1,
        curve: Curves.easeInOutCubicEmphasized.flipped,
      ),
    );
}

class ShapeAnimation extends CurvedAnimation {
  ShapeAnimation({required super.parent})
    : super(
        curve: const Interval(
          2 / 5,
          3 / 5,
          curve: Curves.easeInOutCubicEmphasized,
        ),
      );
}

class SizeAnimation extends CurvedAnimation {
  SizeAnimation({required super.parent})
    : super(
        curve: const Interval(
          0 / 5,
          3 / 5,
          curve: Curves.easeInOutCubicEmphasized,
        ),
        reverseCurve: Interval(
          2 / 5,
          1,
          curve: Curves.easeInOutCubicEmphasized.flipped,
        ),
      );
}

```

開發這些曲線需要疊代，而 Flutter 的熱重載功能可大幅簡化這項作業。如要使用這些動畫，您需要一些轉場效果。在 `lib` 目錄中建立名為 `transitions` 的子目錄，並新增名為 `bottom_bar_transition.dart` 的檔案，加入下列程式碼：

[lib/transitions/bottom_bar_transition.dart](#)

```

import 'package:flutter/material.dart';
import '../animations.dart';

class BottomBarTransition extends StatefulWidget {
  const BottomBarTransition({
    super.key,
    required this.animation,
    required this.backgroundColor,
    required this.child,
  });

  final Animation<double> animation;
  final Color backgroundColor;
  final Widget child;

  @override
  State<BottomBarTransition> createState() => _BottomBarTransition();
}

class _BottomBarTransition extends State<BottomBarTransition> {
  late final Animation<Offset> offsetAnimation = Tween<Offset>(
    begin: const Offset(0, 1),
    end: Offset.zero,
  ).animate(OffsetAnimation(parent: widget.animation));

  late final Animation<double> heightAnimation = Tween<double>(
    begin: 0,
    end: 1,
  ).animate(SizeAnimation(parent: widget.animation));

  @override
  Widget build(BuildContext context) {
    return ClipRect(
      child: DecoratedBox(
        decoration: BoxDecoration(color: widget.backgroundColor),
        child: Align(
          alignment: Alignment.topLeft,
          heightFactor: heightAnimation.value,
          child: FractionalTranslation(
            translation: offsetAnimation.value,
            child: widget.child,
          ),
        ),
      ),
    );
  }
}

```

在 `lib/transitions` 目錄中新增名為 `nav_rail_transition.dart` 的檔案，並加入下列程式碼：

[lib/transitions/nav_rail_transition.dart](#)


```

import 'package:flutter/material.dart';
import '../animations.dart';

class NavRailTransition extends StatefulWidget {
  const NavRailTransition({
    super.key,
    required this.animation,
    required this.backgroundColor,
    required this.child,
  });

  final Animation<double> animation;
  final Widget child;
  final Color backgroundColor;

  @override
  State<NavRailTransition> createState() => _NavRailTransitionState();
}

class _NavRailTransitionState extends State<NavRailTransition> {
  // The animations are only rebuilt by this method when the text
  // direction changes because this widget only depends on Directionality.
  late final bool ltr = Directionality.of(context) == TextDirection.ltr;
  late final Animation<Offset> offsetAnimation = Tween<Offset>(
    begin: ltr ? const Offset(-1, 0) : const Offset(1, 0),
    end: Offset.zero,
  ).animate(OffsetAnimation(parent: widget.animation));
  late final Animation<double> widthAnimation = Tween<double>(
    begin: 0,
    end: 1,
  ).animate(SizeAnimation(parent: widget.animation));

  @override
  Widget build(BuildContext context) {
    return ClipRect(
      child: DecoratedBox(
        decoration: BoxDecoration(color: widget.backgroundColor),
        child: AnimatedBuilder(
          animation: widthAnimation,
          builder: (context, child) {
            return Align(
              alignment: Alignment.topLeft,
              widthFactor: widthAnimation.value,
              child: FractionalTranslation(
                translation: offsetAnimation.value,
                child: widget.child,
              ),
            );
          },
        ),
      ),
    );
  }
}

```

```
}  
}
```

這兩個轉場效果小工具會包裝導覽邊欄和導覽列小工具，以動畫方式呈現這些小工具的顯示與消失。如要使用這兩個轉場效果小工具，請更新兩個小工具，從 `disappearing_bottom_navigation_bar.dart` 開始：

lib/widgets/disappearing_bottom_navigation_bar.dart

```
import 'package:flutter/material.dart';  
  
import '../animations.dart';           // Add this import  
import '../destinations.dart';  
import '../transitions/bottom_bar_transition.dart'; // Add this import  
  
class DisappearingBottomNavigationBar extends StatelessWidget {  
  const DisappearingBottomNavigationBar({  
    super.key,  
    required this.barAnimation,           // Add this parameter  
    required this.selectedIndex,  
    this.onDestinationSelected,  
  });  
  
  final BarAnimation barAnimation;        // Add this variable  
  final int selectedIndex;  
  final ValueChanged<int>? onDestinationSelected;  
  
  @override  
  Widget build(BuildContext context) {  
                                     // Modify from here...  
  
    return BottomBarTransition(  
      animation: barAnimation,  
      backgroundColor: Colors.white,  
      child: NavigationBar(  
        elevation: 0,  
        backgroundColor: Colors.white,  
        destinations: destinations.map<NavigationDestination>((d) {  
          return NavigationDestination(icon: Icon(d.icon), label: d.label);  
        }).toList(),  
        selectedIndex: selectedIndex,  
        onDestinationSelected: onDestinationSelected,  
      ),  
    );  
                                     // ... to here.  
  
  }  
}
```

先前的修改作業會新增其中一個動畫，並整合轉場效果。您可以藉此控制導覽列的顯示和隱藏方式。

接著，請按照下列方式修改 `disappearing_navigation_rail.dart`：

lib/widgets/disappearing_navigation_rail.dart

```

import 'package:flutter/material.dart';

import '../animations.dart';           // Add this import
import '../destinations.dart';
import '../transitions/nav_rail_transition.dart'; // Add this import
import 'animated_floating_action_button.dart'; // Add this import

class DisappearingNavigationRail extends StatelessWidget {
  const DisappearingNavigationRail({
    super.key,
    required this.railAnimation,           // Add this parameter
    required this.railFabAnimation,        // Add this parameter
    required this.backgroundColor,
    required this.selectedIndex,
    this.onDestinationSelected,
  });

  final RailAnimation railAnimation;           // Add this variable
  final RailFabAnimation railFabAnimation;     // Add this variable
  final Color backgroundColor;
  final int selectedIndex;
  final ValueChanged<int>? onDestinationSelected;

  @override
  Widget build(BuildContext context) {
    // Delete colorScheme

    // Modify from here ...

    return NavRailTransition(
      animation: railAnimation,
      backgroundColor: backgroundColor,
      child: NavigationRail(
        selectedIndex: selectedIndex,
        backgroundColor: backgroundColor,
        onDestinationSelected: onDestinationSelected,
        leading: Column(
          children: [
            IconButton(onPressed: () {}, icon: const Icon(Icons.menu)),
            const SizedBox(height: 8),
            AnimatedFloatingActionButton(
              animation: railFabAnimation,
              elevation: 0,
              onPressed: () {},
              child: const Icon(Icons.add),
            ),
          ],
        ),
        groupAlignment: -0.85,
        destinations: destinations.map((d) {
          return NavigationRailDestination(
            icon: Icon(d.icon),
            label: Text(d.label),
          );
        });

```

```
        }).toList(),
      ),
    );

    // ... to here.
  }
}
```

輸入先前的程式碼時，您可能收到一組有關未定義小工具的錯誤警告 - `FloatingActionButton`。如要修正這個問題，請在 `lib/widgets` 中新增名為 `animated_floating_action_button.dart` 的檔案，並加入下列程式碼：

[lib/widgets/animated_floating_action_button.dart](#)

```

import 'dart:ui';

import 'package:flutter/material.dart';
import '../animations.dart';

class AnimatedFloatingActionButton extends StatefulWidget {
  const AnimatedFloatingActionButton({
    super.key,
    required this.animation,
    this.elevation,
    this.onPressed,
    this.child,
  });

  final Animation<double> animation;
  final VoidCallback? onPressed;
  final Widget? child;
  final double? elevation;

  @override
  State<AnimatedFloatingActionButton> createState() =>
    _AnimatedFloatingActionButton();
}

class _AnimatedFloatingActionButton
  extends State<AnimatedFloatingActionButton> {
  late final ColorScheme _colorScheme = Theme.of(context).colorScheme;
  late final Animation<double> _scaleAnimation = ScaleAnimation(
    parent: widget.animation,
  );
  late final Animation<double> _shapeAnimation = ShapeAnimation(
    parent: widget.animation,
  );

  @override
  Widget build(BuildContext context) {
    return ScaleTransition(
      scale: _scaleAnimation,
      child: FloatingActionButton(
        elevation: widget.elevation,
        shape: RoundedRectangleBorder(
          borderRadius: BorderRadius.all(
            Radius.circular(lerpDouble(30, 15, _shapeAnimation.value!)),
          ),
        ),
        backgroundColor: _colorScheme.tertiaryContainer,
        foregroundColor: _colorScheme.onTertiaryContainer,
        onPressed: widget.onPressed,
        child: widget.child,
      ),
    );
  }
}

```

```
}  
}
```

如要將這些變更帶入應用程式，請按照下列方式更新 `main.dart` 檔案：

[lib/main.dart](#)

```

import 'package:flutter/material.dart';

import 'animations.dart'; // Add this import
import 'models/data.dart' as data;
import 'models/models.dart';
import 'widgets/animated_floating_action_button.dart'; // Add this import
import 'widgets/disappearing_bottom_navigation_bar.dart';
import 'widgets/disappearing_navigation_rail.dart';
import 'widgets/email_list_view.dart';

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData.light(),
      home: Feed(currentUser: data.user_0),
    );
  }
}

class Feed extends StatefulWidget {
  const Feed({super.key, required this.currentUser});

  final User currentUser;

  @override
  State<Feed> createState() => _FeedState();
}

class _FeedState extends State<Feed> with SingleTickerProviderStateMixin {
  late final _colorScheme = Theme.of(context).colorScheme;
  late final _backgroundColor = Color.alphaBlend(
    _colorScheme.primary.withAlpha(36),
    _colorScheme.surface,
  );

  // Add from here...

  late final _controller = AnimationController(
    duration: const Duration(milliseconds: 1000),
    reverseDuration: const Duration(milliseconds: 1250),
    value: 0,
    vsync: this,
  );

  late final _railAnimation = RailAnimation(parent: _controller);
  late final _railFabAnimation = RailFabAnimation(parent: _controller);
  late final _barAnimation = BarAnimation(parent: _controller);

  // ... to here.

```

```

int selectedIndex = 0;
// Remove wideScreen
bool controllerInitialized = false; // Add this variable

@override
void didChangeDependencies() {
  super.didChangeDependencies();

  final double width = MediaQuery.of(context).size.width;
  // Remove wideScreen reference // Add from here ...

  final AnimationStatus status = _controller.status;
  if (width > 600) {
    if (status != AnimationStatus.forward &&
        status != AnimationStatus.completed) {
      _controller.forward();
    }
  } else {
    if (status != AnimationStatus.reverse &&
        status != AnimationStatus.dismissed) {
      _controller.reverse();
    }
  }
  if (!controllerInitialized) {
    controllerInitialized = true;
    _controller.value = width > 600 ? 1 : 0;
  }

  // ... to here.
}

// Add from here ...

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

// ... to here.

@override
Widget build(BuildContext context) {

  // Modify from here ...

  return AnimatedBuilder(
    animation: _controller,
    builder: (context, _) {
      return Scaffold(
        body: Row(
          children: [
            DisappearingNavigationRail(
              railAnimation: _railAnimation,
              railFabAnimation: _railFabAnimation,
              selectedIndex: selectedIndex,

```



```

        backgroundColor: _backgroundColor,
        onDestinationSelected: (index) {
          setState(() {
            selectedIndex = index;
          });
        },
      ),
    Expanded(
      child: Container(
        color: _backgroundColor,
        child: EmailListView(
          selectedIndex: selectedIndex,
          onSelected: (index) {
            setState(() {
              selectedIndex = index;
            });
          },
          currentUser: widget.currentUser,
        ),
      ),
    ),
  ],
),
floatingActionButton: AnimatedFloatingActionButton(
  animation: _barAnimation,
  onPressed: () {},
  child: const Icon(Icons.add),
),
bottomNavigationBar: DisappearingBottomNavigationBar(
  barAnimation: _barAnimation,
  selectedIndex: selectedIndex,
  onDestinationSelected: (index) {
    setState(() {
      selectedIndex = index;
    });
  },
),
);
},
);
// ... to here.
}
}

```

執行應用程式。一開始，應用程式看起來應該與先前相同。調整畫面大小，即可查看 UI 如何根據大小和尺寸，在導覽邊欄和導覽列之間切換。現在這些轉場效果的動作應該會呈現流暢且活潑的感覺。使用熱重載變更動畫曲線，看看應用程式的風格有何變化。

步驟 8 · 約 5 分鐘

8. 新增詳細資料檢視畫面清單

此外，訊息應用程式很適合用來展示清單/詳細資料版面配置，但前提是螢幕寬度要夠大。首先，在 `lib/widgets` 中新增名為 `reply_list_view.dart` 的檔案，並填入下列程式碼：

`lib/widgets/reply_list_view.dart`

```
import 'package:flutter/material.dart';

import '../models/data.dart' as data;
import 'email_widget.dart';

class ReplyListView extends StatelessWidget {
  const ReplyListView({super.key});

  @override
  Widget build(BuildContext context) {
    return Padding(
      padding: const EdgeInsets.only(right: 8.0),
      child: ListView(
        children: [
          const SizedBox(height: 8),
          ...List.generate(data.replies.length, (index) {
            return Padding(
              padding: const EdgeInsets.only(bottom: 8.0),
              child: EmailWidget(
                email: data.replies[index],
                isPreview: false,
                isThreaded: true,
                showHeadline: index == 0,
              ),
            );
          }),
        ],
      ),
    );
  }
}
```

接著，在 `lib/transitions` 中新增 `list_detail_transition.dart`，並填入下列程式碼：

`lib/transitions/list_detail_transition.dart`

```

import 'dart:ui';

import 'package:flutter/material.dart';
import '../animations.dart';

class ListDetailTransition extends StatefulWidget {
  const ListDetailTransition({
    super.key,
    required this.animation,
    required this.one,
    required this.two,
  });

  final Animation<double> animation;
  final Widget one;
  final Widget two;

  @override
  State<ListDetailTransition> createState() => _ListDetailTransitionState();
}

class _ListDetailTransitionState extends State<ListDetailTransition> {
  Animation<double> widthAnimation = const AlwaysStoppedAnimation(0);
  late final Animation<double> sizeAnimation = SizeAnimation(
    parent: widget.animation,
  );
  late final Animation<Offset> offsetAnimation = Tween<Offset>(
    begin: const Offset(1, 0),
    end: Offset.zero,
  ).animate(OffsetAnimation(parent: sizeAnimation));
  double currentFlexFactor = 0;

  @override
  void didChangeDependencies() {
    super.didChangeDependencies();

    final double width = MediaQuery.of(context).size.width;
    double nextFlexFactor = switch (width) {
      >= 800 && < 1200 => lerpDouble(1000, 2000, (width - 800) / 400)!,
      >= 1200 && < 1600 => lerpDouble(2000, 3000, (width - 1200) / 400)!,
      >= 1600 => 3000,
      _ => 1000,
    };

    if (nextFlexFactor == currentFlexFactor) {
      return;
    }

    if (currentFlexFactor == 0) {
      widthAnimation = Tween<double>(
        begin: 0,
        end: nextFlexFactor,

```

```

    ).animate(sizeAnimation);
  } else {
    final TweenSequence<double> sequence = TweenSequence([
      if (sizeAnimation.value > 0) ...[
        TweenSequenceItem(
          tween: Tween(begin: 0, end: widthAnimation.value),
          weight: sizeAnimation.value,
        ),
      ],
      if (sizeAnimation.value < 1) ...[
        TweenSequenceItem(
          tween: Tween(begin: widthAnimation.value, end: nextFlexFactor),
          weight: 1 - sizeAnimation.value,
        ),
      ],
    ]);

    widthAnimation = sequence.animate(sizeAnimation);
  }

  currentFlexFactor = nextFlexFactor;
}

@override
Widget build(BuildContext context) {
  return widthAnimation.value.toInt() == 0
    ? widget.one
    : Row(
      children: [
        Flexible(flex: 1000, child: widget.one),
        Flexible(
          flex: widthAnimation.value.toInt(),
          child: FractionalTranslation(
            translation: offsetAnimation.value,
            child: widget.two,
          ),
        ),
      ],
    );
}
}

```

如要將這項內容整合到應用程式中，請按照下列方式更新 `lib/main.dart`：

[lib/main.dart](#)

```

import 'package:flutter/material.dart';

import 'animations.dart';
import 'models/data.dart' as data;
import 'models/models.dart';
import 'transitions/list_detail_transition.dart';          // Add import
import 'widgets/animated_floating_action_button.dart';
import 'widgets/disappearing_bottom_navigation_bar.dart';
import 'widgets/disappearing_navigation_rail.dart';
import 'widgets/email_list_view.dart';
import 'widgets/reply_list_view.dart';                    // Add import

void main() {
  runApp(const MainApp());
}

class MainApp extends StatelessWidget {
  const MainApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      theme: ThemeData.light(),
      home: Feed(currentUser: data.user_0),
    );
  }
}

class Feed extends StatefulWidget {
  const Feed({super.key, required this.currentUser});

  final User currentUser;

  @override
  State<Feed> createState() => _FeedState();
}

class _FeedState extends State<Feed> with SingleTickerProviderStateMixin {
  late final _colorScheme = Theme.of(context).colorScheme;
  late final _backgroundColor = Color.alphaBlend(
    _colorScheme.primary.withAlpha(36),
    _colorScheme.surface,
  );

  late final _controller = AnimationController(
    duration: const Duration(milliseconds: 1000),
    reverseDuration: const Duration(milliseconds: 1250),
    value: 0,
    vsync: this,
  );

  late final _railAnimation = RailAnimation(parent: _controller);
  late final _railFabAnimation = RailFabAnimation(parent: _controller);
  late final _barAnimation = BarAnimation(parent: _controller);

```

```

int selectedIndex = 0;
bool controllerInitialized = false;

@override
void didChangeDependencies() {
  super.didChangeDependencies();

  final double width = MediaQuery.of(context).size.width;
  final AnimationStatus status = _controller.status;
  if (width > 600) {
    if (status != AnimationStatus.forward &&
        status != AnimationStatus.completed) {
      _controller.forward();
    }
  } else {
    if (status != AnimationStatus.reverse &&
        status != AnimationStatus.dismissed) {
      _controller.reverse();
    }
  }
  if (!controllerInitialized) {
    controllerInitialized = true;
    _controller.value = width > 600 ? 1 : 0;
  }
}

@override
void dispose() {
  _controller.dispose();
  super.dispose();
}

@override
Widget build(BuildContext context) {
  return AnimatedBuilder(
    animation: _controller,
    builder: (context, _) {
      return Scaffold(
        body: Row(
          children: [
            DisappearingNavigationRail(
              railAnimation: _railAnimation,
              railFabAnimation: _railFabAnimation,
              selectedIndex: selectedIndex,
              backgroundColor: _backgroundColor,
              onDestinationSelected: (index) {
                setState(() {
                  selectedIndex = index;
                });
              },
            ),
          ],
        ),
      );
    },
  );
}

```

```

Expanded(
  child: Container(
    color: _backgroundColor,

                                // Update from here ...

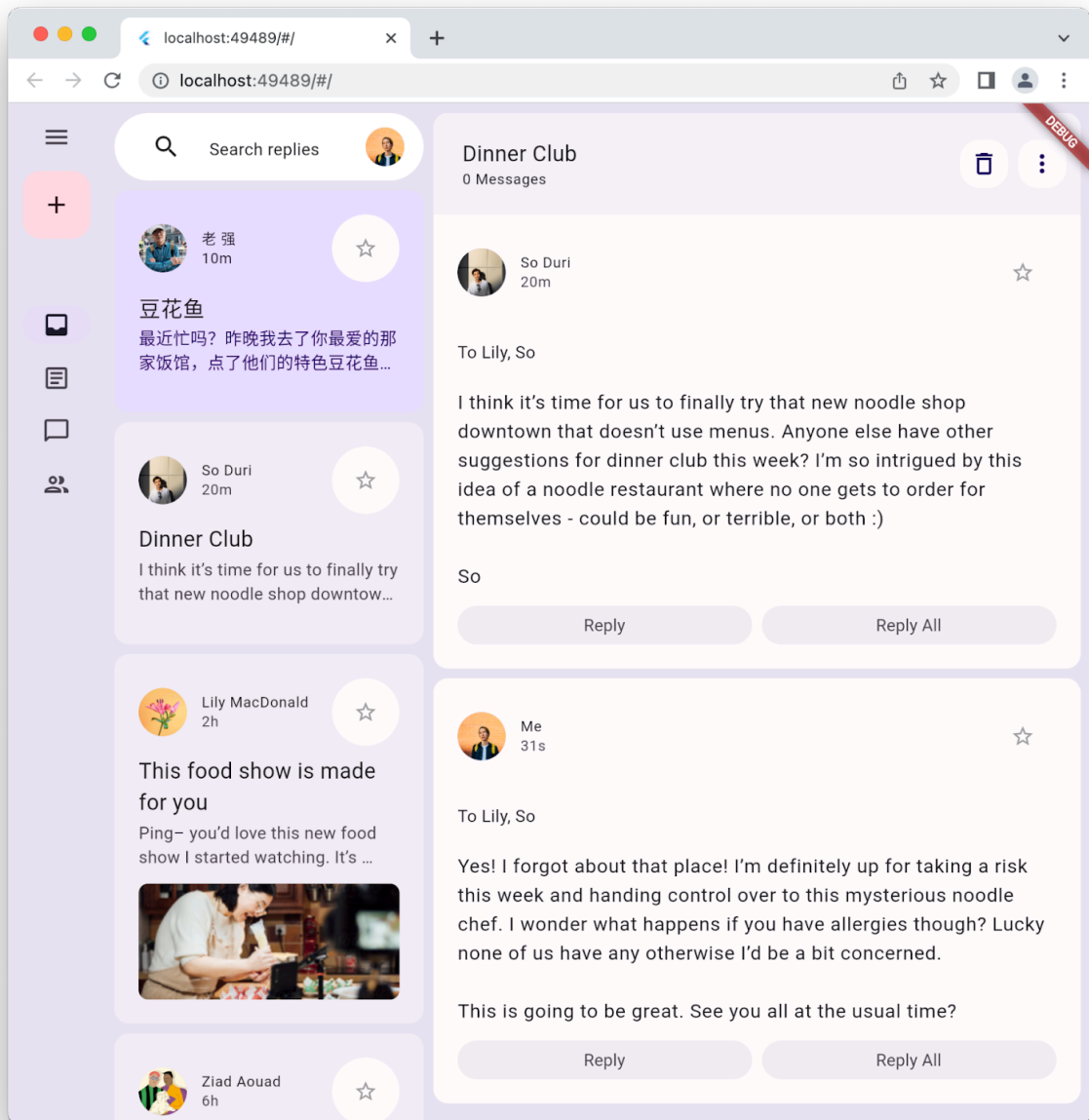
    child: ListDetailTransition(
      animation: _railAnimation,
      one: EmailListView(
        selectedIndex: selectedIndex,
        onSelected: (index) {
          setState(() {
            selectedIndex = index;
          });
        },
        currentUser: widget.currentUser,
      ),
      two: const ReplyListView(),
    ),

                                // ... to here.

  ),
),
],
),
floatingActionButton: AnimatedFloatingActionButton(
  animation: _barAnimation,
  onPressed: () {},
  child: const Icon(Icons.add),
),
bottomNavigationBar: DisappearingBottomNavigationBar(
  barAnimation: _barAnimation,
  selectedIndex: selectedIndex,
  onDestinationSelected: (index) {
    setState(() {
      selectedIndex = index;
    });
  },
),
);
},
);
}
}

```

執行應用程式，查看所有整合內容。您在代表實際應用程式的應用程式中，擁有 Material 3 樣式，以及不同版面配置之間的動畫。畫面應如下所示：



步驟 9 · 約 0 分鐘

9. 恭喜

恭喜，您已成功建構第一個 Material 3 Flutter 應用程式！

如要查看本程式碼研究室的所有步驟，請前往 [Flutter 程式碼研究室 GitHub 存放區](#)。

後續步驟

查看一些程式碼研究室...

- [使用 Flutter 和 Flame 建構遊戲](#)
- [讓 Flutter 應用程式從無趣變為美觀](#)
- [在 Flutter 外掛程式中使用 FFI](#)

其他資訊

- 進一步瞭解 [Flutter 中的 Material 3 更新](#)
 - 請參閱 [Flutter Material 3 說明文件](#)
 - 查看所有 [Material 3 資源](#)
-